

SD Basic - Data Conversion

Dates, times, numbers and text between the form SD stores and the form people read.

SD stores a date as a day number and a time as a second count. Neither is readable, and neither is what a user types. `iconv()` turns what a person wrote into what SD stores; `oconv()` turns it back. `fmt()` lays a value out in a fixed width. This page covers all three, and the character-level conversions beside them.

SD folds case, so a program may be written in either case. Keywords and conversion codes are shown here in lower case; **the codes themselves are not case sensitive**. In the tables, *italics* mark something you supply.

The two directions

```
iconv(expression, conversion)
oconv(expression, conversion)
```

`iconv()`**in** — external form to internal. Use it on input`oconv()`**out** — internal to external. Use it on output

`iconvs()` and `oconvs()` do the same to every element of a dynamic array.

A failed `iconv()` returns the null string and sets `status()`. `iconv('nonsense', 'D')` returns empty and `status()` afterwards is **1**.

```
d = iconv(typed.date, 'D')
if status() then
  print 'That is not a date.'
end
```

This is the validation idiom, and the return value alone is not enough — a user who typed nothing also gets an empty result, so testing the result cannot tell an empty input from a bad one. `status()` can.

Dates

```
iconv(text, 'D')
oconv(daynumber, 'D{n}{separator}{format}')
```

Day zero is 31 December 1967. `oconv(0, 'D4/')` is `12/31/1967`. Dates before that are negative. `26 AUG 2026` is day **21423**.

CONVERSION	RESULT FOR DAY 21423
<code>oconv(d, 'D')</code>	<code>26 AUG 2026</code>
<code>oconv(d, 'D2/')</code>	<code>08/26/26</code>
<code>oconv(d, 'D4-')</code>	<code>08-26-2026</code>
<code>oconv(d, 'DMA')</code>	<code>AUGUST</code>
<code>oconv(d, 'DW')</code>	<code>3</code> — day of the week
<code>oconv(d, 'DWA')</code>	<code>WEDNESDAY</code>
<code>oconv(d, 'DQ')</code>	<code>3</code> — quarter
<code>oconv(d, 'DY')</code>	<code>2026</code>

The digit after `D` is how many digits of year to show; the character after that is the separator.

The default order is month, day, year. `D2/` gives `08/26/26` for the 26th of August. `E` **puts it in day-month-year order:** `D2/E` gives `26/08/26` and `D4/E` gives `26/08/2026`. A site that expects day-first must say `E` on **every** conversion; a report that mixes the two is a formatting bug that reads as a data bug.

`iconv()` accepts a wide range of typed forms — `26 AUG 2026`, `08/26/2026`, `26-8-26` — and resolves a two-digit year against a sliding window. **Always check** `status()`: `13/13/26` is not a date and will come back empty.

Times

```
iconv(text, 'MT')
oconv(seconds, 'MT{H}{S}')
```

A time is the number of seconds since midnight. `14:30:05` is **52205**.

CONVERSION	RESULT
<code>oconv(t, 'MT')</code>	<code>14:30</code>
<code>oconv(t, 'MTS')</code>	<code>14:30:05</code>
<code>oconv(t, 'MTH')</code>	<code>02:30pm</code>
<code>oconv(t, 'MTHS')</code>	<code>02:30:05pm</code>

`s` adds seconds, `h` switches to a 12-hour clock. `date()` and `time()` give today's day number and the current second count; `timedate()` gives both, already formatted.

Numbers

SCALING AND SEPARATORS

```
oconv(number, 'MD{n}{separator}')
```

`md` *n* moves the decimal point *n* places **left**, so the stored value is an integer and the displayed one has decimals.

CONVERSION	RESULT
<code>oconv(1234567, 'MD2')</code>	<code>12345.67</code>
<code>oconv(1234567, 'MD2,')</code>	<code>12,345.67</code>
<code>oconv(1234567, 'MD0,')</code>	<code>1,234,567</code>
<code>oconv(-42, 'MD2')</code>	<code>-0.42</code>

MD2 divides by 100 — it does not round to two places. `oconv(-42, 'MD2')` is `-0.42`, not `-42.00`. It is for money held as whole pence or cents. **To show two decimals of a value that is already scaled, use `fmt()` with a `2` after the justification.**

OTHER BASES

CONVERSION	RESULT
<code>oconv(255, 'MX')</code>	<code>FF</code>
<code>iconv('FF', 'MX')</code>	<code>255</code>
<code>oconv(5, 'MB')</code>	<code>101</code>
<code>oconv(8, 'MO')</code>	<code>10</code>

`mx` hexadecimal, `mb` binary, `mo` octal. `xtd()` and `dtx()` do the same for hex without going through `oconv()` — see *Characters and codes* below.

SELECTING CHARACTERS

CONVERSION	RESULT
<code>oconv('abc', 'MCU')</code>	<code>ABC</code>
<code>oconv('ABC', 'MCL')</code>	<code>abc</code>
<code>oconv('hello world', 'MCT')</code>	<code>Hello World</code>
<code>oconv('a1b2', 'MCN')</code>	<code>12</code> — digits only
<code>oconv('a1b2', 'MCA')</code>	<code>ab</code> — letters only

`mcn` and `mca` **discard** everything else rather than replacing it, which makes them a quick way to strip formatting out of a typed reference.

Laying a value out: `fmt()`

```
fmt(expression, specification)
fmts(array, specification)
```

The specification is a width, a justification, and optional extras.

CALL	RESULT
<code>fmt('ab', '10L')</code>	<code>ab_____</code> — left, padded to 10

CALL	RESULT
<code>fmt('ab', '10R')</code>	<code> ab</code> — right
<code>fmt('ab', '10*R')</code>	<code>*****ab</code> — padded with <code>*</code>
<code>fmt(1234.5, '12R2')</code>	<code> 1234.50</code> — two decimals, right

The character before the justification letter is the fill character; a digit after it is the number of decimal places.

A value longer than the width is wrapped, not truncated. `fmt('abcdefgh', '4L')` returns `abcd` and `efgh` separated by a **text mark**, not `abcd`. A field written straight to a screen or a file therefore gains a stray character rather than being cut short. **To truncate, take a substring first** — `fmt(s[1,4], '4L')`.

THE FORMAT QUALIFIER

The same thing can be written by placing the specification **straight after the value, with nothing between them**:

```
print total '10r2'
```

With `t` holding `1234.5`, `t '10R2'` gives `1234.50`. **There is no `fmt` keyword in this form** — writing `t fmt '10R2'` compiles `fmt` as a variable name and aborts at run time with *"Unassigned variable FMT"*. The qualifier is recognised by juxtaposition alone, which is why it is allowed only where an expression cannot be followed by something else.

Characters and codes

CALL	RESULT
<code>char(65)</code>	<code>A</code>
<code>seq('A')</code>	<code>65</code>
<code>seq('')</code>	<code>0</code>
<code>xtd('FF')</code>	<code>255</code>
<code>dtx(255)</code>	<code>ff</code>
<code>dtx(255, 4)</code>	<code>00ff</code>

<code>char()</code>	the character with the given code
<code>seq()</code>	the code of the first character. <code>seq('')</code> is <code>0</code> , the same as <code>seq(char(0))</code>
<code>xtd()</code>	hexadecimal text to a number
<code>dtx()</code>	a number to hexadecimal text, in lower case , optionally zero-padded to a width
<code>ascii()</code> · <code>ebcdic()</code>	translate a string between the two character sets

`ascii(char(193))` is `A` — EBCDIC 193 is the letter A.

`char()` **is how you write a mark character or a control code as a constant**, and `equate` is the place to put it:

```
equate esc to char(27)
equate fm to char(254)
```

Splitting a delimited string

```
dparse string, delimiter, var1, var2, ...
```

Splits *string* on *delimiter* and assigns the pieces to the named variables in order. It is the fixed-shape alternative to a `remove` loop: use it when you know how many pieces there are and want them in named variables rather than an array.

Missing pieces leave their variables empty; extra pieces are discarded.

`dparse.csv` does the same for a CSV line, honouring quoting — see SD Basic - CSV Files.

Dictionary conversions

```
itype(dictionary.record)
```

Evaluates a dictionary item — an I-type expression — against the record currently being processed, and returns what it computes. It is how a program uses the same derived field a query would, instead of duplicating the formula.

`trans()`, `rtrans()` and `xlate()` fetch a field from another file and can apply that file's own conversion; they are covered in SD Basic - File Handling because they open a file to do it.

National settings

```
getnls(key)
setnls key, value
```

Report and set the national-language settings — currency symbol, thousands and decimal separators, date order. `setnls` **changes them for the session**, so a program that alters them and then aborts leaves the session changed. Read the old value, set the new, and restore it on the way out.

What is not here

Nothing in the conversion group has been removed from this port.

`fmt()`, `char()`, `seq()`, `ascii()`, `ebcdic()` and the `convert` statement are also mentioned in SD Basic - String Functions, where they are used on text rather than for conversion. `precision`, which decides how many decimal places a floating-point number shows when it becomes a string, is in SD Basic - Math Functions.

See also

SD Basic - String Functions · SD Basic - Math Functions · SD Basic - File Handling · SD Basic - CSV Files.
